TRIP PLANNER PWA · FICHE MODULE · MEMBRE 4

Module 4 — Gestion des dépenses

Ce que fait le module, la logique métier qu'il porte, et ce que vous devez savoir présenter.

COLLECTION MONGODB	FONCTIONNALITÉ AVANCÉE	RÔLE TRANSVERSE
expenses	**Conversion multi-devises et graphiques de budget**	—

1. Objectif du module

Suivre les dépenses partagées du groupe : saisie dans n'importe quelle devise, conversion automatique dans la devise du voyage, répartition entre participants, photo du reçu, suivi du budget avec alerte à 80 %, graphiques, et calcul de « qui doit combien à qui ».

> Règle d'équipe : vous êtes le seul à écrire dans votre collection. Les autres modules passent par votre API (ou votre service), et toute modification d'un modèle partagé demande l'accord du groupe.

2. Modèle de données

Champ	Type	Règle / rôle
`trip`	ref trips	obligatoire
`label`	texte	1 à 120 caractères
`amount, currency`	nombre, ISO 4217	montant d'origine (> 0) et devise d'origine
`amountInBase`	nombre	montant converti dans la devise du voyage
`rateUsed`	nombre	1 unité de la devise = rateUsed unités de la devise du voyage
`category`	enum	accommodation, transport, food, activities, shopping, health, other
`date`	date	jour de la dépense
`paidBy`	ref users	payeur (membre du voyage)
`splitBetween[]`	refs users	participants qui partagent la dépense
`receiptUrl`	texte | null	photo du reçu

3. Logique métier

Saisie

- Par défaut, le **payeur** est l'utilisateur connecté et la dépense est **partagée entre tous les membres**.
- Le payeur et chaque participant doivent être **membres du voyage** (sinon 400).
- La dépense est **stockée deux fois** : montant d'origine et montant converti, avec le taux utilisé. Le budget reste donc juste même si les taux changent ensuite.
- La conversion est recalculée seulement si le montant ou la devise change.
- Arrondis : montants à 2 décimales, taux à 6 décimales.

- Création idempotente (id client), modification en last-write-wins, suppression idempotente.

Taux de change

- Les taux sont récupérés depuis une API publique (open.er-api.com) avec la devise du voyage comme base, puis **mis en cache 24 h côté serveur**.
- Si l'API est indisponible : **table de secours** de 34 devises (base EUR), et nouvel essai au bout d'une heure.
- Devise inconnue → 400 avec une erreur sur le champ `currency`.
- `GET /rates` est public : le client garde les derniers taux pour faire des estimations hors ligne.

Budget, soldes et remboursements

- Total dépensé = somme des `amountInBase` ; pourcentage = total / budget.
- **Alerte à 80 %** du budget (orange), et « Budget exceeded » au-delà de 100 % (rouge).
- Répartition par catégorie et par jour pour les graphiques.
- **Soldes** : le payeur est crédité du montant, chaque participant est débité de sa part égale (montant / nombre de participants).
- **Remboursements** : algorithme glouton — le plus gros débiteur rembourse le plus gros créancier, et ainsi de suite, ce qui limite le nombre de virements.

Hors ligne

- Le résumé est recalculé dans le navigateur avec le même algorithme : il fonctionne sans réseau.
- Une dépense en devise étrangère saisie hors ligne est convertie avec les **derniers taux en cache** et affiche le badge « estimate » ; le serveur recalcule la vraie valeur à la synchronisation.
- La photo du reçu nécessite une connexion (envoi d'image).

4. Fonctionnalité avancée — Conversion multi-devises et graphiques de budget

Conversion automatique à la saisie, avec double stockage (devise d'origine et devise du voyage).

Taux mis en cache une fois par jour côté serveur, avec repli en cas de panne.

Graphiques (Recharts) : **camembert par catégorie** et **histogramme par jour**, avec une description textuelle pour les lecteurs d'écran.

Barre de progression du budget avec les seuils 80 % et 100 %.

Tableau « To settle up » : qui doit combien à qui.

5. API REST (préfixe `/api/v1`, réponse `{ data, message, errors }`)

Méthode	Route	Accès	Rôle
`GET / POST`	`/trips/:tripId/expenses`	membre	liste (catégorie, période, tri) / ajouter
`GET / PATCH / DELETE`	`/trips/:tripId/expenses/:expenseId`	membre	détail, modifier, supprimer
`GET`	`/trips/:tripId/expenses/summary`	membre	total, %, alerte, graphiques, soldes
`GET`	`/rates?base=EUR`	public	taux du jour
`POST`	`/uploads?folder=receipts`	membre	photo du reçu (module 6)

6. Écrans (front Next.js)

- Onglet « Budget » : total, progression, alerte, remboursements, deux graphiques
- Liste des dépenses avec filtre par catégorie ; formulaire avec devise, payeur, participants et photo du reçu

7. Fichiers dont vous êtes responsable

API — trip-planner-api

- `src/modules/expenses/ (expense.model, expense.routes, rates.service)`

Front — trip-planner-web

- `src/app/trips/[id]/expenses/page.tsx`
- `src/components/trips/ExpenseCharts.tsx`
- `fonctions loadRates / estimateInBase dans src/lib/sync.ts`

8. Liens avec les autres modules

- **Module 2** : budget, devise de référence, liste des membres, file hors ligne.
- **Module 6** : envoi des photos de reçus via `/uploads`.

9. À démontrer à la soutenance

- ☐ Ajouter une dépense en TND sur un voyage en EUR : montrer le taux et le montant converti.
- ☐ Dépasser 80 % du budget : l'alerte apparaît.
- ☐ Dépense payée par un participant et partagée par deux : montrer le remboursement calculé.
- ☐ Hors ligne : dépense en USD → badge « estimate », puis valeur exacte après synchronisation.

10. Reste à faire

- **Tests unitaires de la conversion** (exigés par le cahier des charges) : même devise, devise inconnue, arrondis, table de secours.
- Tests d'intégration des endpoints ; requêtes Postman.

11. Questions probables du jury

Pourquoi stocker le montant converti ?
Pour que le budget ne change pas quand les taux évoluent, et pour connaître le taux réellement appliqué.

Comment est calculé « qui doit à qui » ?
Par les soldes (payé − dû), puis un algorithme glouton qui associe débiteurs et créanciers.